```
%% ========================================================================
%  BILAYER SID EPIDEMIOLOGICAL-ACTUARIAL MODEL
%  HIV/AIDS Insurance — Four-Product Suite, Indonesia
% ========================================================================

clear; clc; close all;
rng(2025);   % global reproducibility seed

%% ========================================================================
%  SECTION 1: DATA
% ========================================================================
times     = (1:24)';
year_base = 2000;
year_obs  = year_base + (times - 1);

% UNAIDS Indonesia HIV estimates
plhiv_raw = [42000, 74000, 110000, 150000, 190000, 230000, 280000, 320000, ...
             350000, 390000, 420000, 450000, 480000, 500000, 520000, ...
             530000, 540000, 550000, 560000, 560000, 570000, 570000, ...
             570000, 570000]';

death_raw = [770, 1200, 2000, 3100, 4400, 6100, 7600, 9100, 11000, 13000, ...
             15000, 17000, 20000, 22000, 24000, 25000, 26000, 26000, ...
             27000, 27000, 27000, 27000, 27000, 24000]';

% Population (BPS linear interpolation 2000-2023)
Pop = linspace(210e6, 278e6, 24)';

% Convert to proportions (normalized state variables)
data_prev   = plhiv_raw ./ Pop;
data_death  = death_raw ./ Pop;
data_target = [data_prev, data_death];

%% ========================================================================
%  SECTION 2: FIXED PARAMETERS & DISEASE-FREE EQUILIBRIUM
% ========================================================================
Lambda    = 0.01707;   % Crude birth rate       (BPS Long-Form SP2020)
mu        = 0.00474;   % Crude death rate        (BPS/UNFPA 2020)
nu        = 0.04237;   % Insurance uptake rate   (JKN proxy, DJSN 2019-2025)
k         = 0.600;     % ART mortality reduction (Wang et al. 2024)
gamma_art = 0.10;      % Viral suppression / infectivity factor (Cohen et al. 2011)

fixed_params = [Lambda, mu, nu, k, gamma_art];

s_star  = Lambda / (Lambda + nu);
sp_star = nu     / (Lambda + nu);

fprintf('========================================\n');
```

```
========================================
```

```
fprintf('  DISEASE-FREE EQUILIBRIUM COORDINATES  \n');
```

```
  DISEASE-FREE EQUILIBRIUM COORDINATES
```

```
fprintf('=======================================\n');
```

```
=====================================
```

```
fprintf('  s*  = %.6f  (uninsured susceptible)\n', s_star);
```

```
  s*  = 0.287180  (uninsured susceptible)
```

```
fprintf('  sp* = %.6f  (insured susceptible)\n',   sp_star);
```

```
  sp* = 0.712820  (insured susceptible)
```

```
fprintf('  s* + sp* = %.6f  [should = 1.0]\n',     s_star + sp_star);
```

```
  s* + sp* = 1.000000  [should = 1.0]
```

```
%% ========================================================================
%  SECTION 3: INITIAL CONDITIONS (year 2000, no insurance yet)
% ========================================================================
i0     = data_prev(1);
x0     = [1-i0; 0; i0; 0];     % [s; sp; i; ip]
x0_mac = x0;

%% ========================================================================
%  SECTION 4: PARAMETER ESTIMATION (multi-start SQP)
% ========================================================================
lb = [0.01,    0,  0.001];
ub = [5.00, 5000,  0.500];

opts_fit = optimoptions('fmincon', ...
    'Algorithm',              'sqp', ...
    'Display',                'none', ...
    'MaxFunctionEvaluations', 8000, ...
    'FunctionTolerance',      1e-12, ...
    'StepTolerance',          1e-12, ...
    'OptimalityTolerance',    1e-10);

n_starts = 50;
best_err = inf;
best_p   = [];

fprintf('\nEstimating parameters (multi-start n=%d)...\n', n_starts);
```

```
Estimating parameters (multi-start n=50)...
```

```
for i = 1:n_starts
    p0 = [lb(1) + rand*(ub(1)-lb(1)), ...
          lb(2) + rand*(ub(2)-lb(2)), ...
          lb(3) + rand*(ub(3)-lb(3))];
    try
        [p_est, err] = fmincon( ...
            @(p) obj_nsse(p, times, x0, fixed_params, data_target), ...
            p0, [], [], [], [], lb, ub, [], opts_fit);
        if err < best_err
            best_err = err;
```

```
            best_p   = p_est;
        end
    catch
    end
end

if isempty(best_p)
    error('Optimization failed on all starts. Check bounds or data.');
end

beta    = best_p(1);
alpha   = best_p(2);
delta   = best_p(3);
delta_p = k * delta;
k_inf   = gamma_art;

fprintf('\n========================================\n');
```

```
========================================
```

```
fprintf('  ESTIMATED PARAMETERS                  \n');
```

```
  ESTIMATED PARAMETERS
```

```
fprintf('========================================\n');
```

```
========================================
```

```
fprintf('  beta  = %.6f  [baseline infection rate]\n',   beta);
```

```
  beta  = 0.642308  [baseline infection rate]
```

```
fprintf('  alpha = %.6f  [behavioral response]\n',       alpha);
```

```
  alpha = 1088.635016  [behavioral response]
```

```
fprintf('  delta = %.6f  [AIDS-specific mortality rate]\n', delta);
```

```
  delta = 0.048601  [AIDS-specific mortality rate]
```

```
fprintf('  NSSE  = %.4e\n',                              best_err);
```

```
  NSSE  = 4.1735e-01
```

```
%% ========================================================================
%  SECTION 5: SIMULATE BEST-FIT MODEL
% ========================================================================
[~, x_fit] = ode45(@(t,y) ode_macro(t,y,best_p,fixed_params), times, x0);

sim_prev  = x_fit(:,3) + x_fit(:,4);
sim_death = delta .* x_fit(:,3) + delta_p .* x_fit(:,4);

%% ========================================================================
%  SECTION 6: BOOTSTRAP CONFIDENCE INTERVALS (residual resampling)
% ========================================================================
n_boot     = 1000;
```

```
param_boot = NaN(n_boot, 3);
res_prev   = data_prev  - sim_prev;
res_death  = data_death - sim_death;

opts_boot = optimoptions('fmincon', ...
    'Algorithm',              'sqp', ...
    'Display',                'none', ...
    'MaxFunctionEvaluations', 5000);

fprintf('Running bootstrap (n=%d)...\n', n_boot);
```

```
Running bootstrap (n=1000)...
```

```
for b = 1:n_boot
    idx     = randi(length(times), length(times), 1);
    prev_b  = max(0, sim_prev  + res_prev(idx));
    death_b = max(0, sim_death + res_death(idx));
    data_b  = [prev_b, death_b];
    try
        p0_b = max(lb, min(ub, best_p .* (1 + 0.10*randn(1,3))));
        p_b  = fmincon( ...
            @(p) obj_nsse(p, times, x0, fixed_params, data_b), ...
            p0_b, [], [], [], [], lb, ub, [], opts_boot);
        param_boot(b,:) = p_b;
    catch
    end
end

valid_rows = ~any(isnan(param_boot), 2);
param_boot = param_boot(valid_rows, :);
n_valid    = size(param_boot, 1);
fprintf('Valid bootstrap samples: %d / %d (%.1f%%)\n', ...
    n_valid, n_boot, 100*n_valid/n_boot);
```

```
Valid bootstrap samples: 1000 / 1000 (100.0%)
```

```


param_ci   = prctile(param_boot, [2.5, 97.5]);
param_mean = mean(param_boot);
param_std  = std(param_boot);
param_min  = min(param_boot);
param_max  = max(param_boot);
param_tags = {'beta', 'alpha', 'delta'};

fprintf('\n=============================================================================
```

```
==========================================================================
```

```
fprintf('  BOOTSTRAP PARAMETER ESTIMATES (n = %d valid)\n', n_valid);
```

```
  BOOTSTRAP PARAMETER ESTIMATES (n = 1000 valid)
```

```
fprintf('=============================================================================
```

```
==========================================================================
```

```
fprintf('  %-8s | %-10s | %-10s | %-10s | %-22s | %-20s\n', ...
    'Param', 'Best-fit', 'Boot. Mean', 'Boot. Std', '95% CI', 'Range (Min - Max)');
```

```
  Param    | Best-fit   | Boot. Mean | Boot. Std  | 95% CI                 | Range (Min - Max)
```

```
fprintf('  %s\n', repmat('-', 1, 95));
```

```
  -----------------------------------------------------------------------------------------------
```

```
for j = 1:3
    fprintf('  %-8s | %-10.4f | %-10.4f | %-10.4f | [%-8.4f, %-8.4f] | [%-8.4f, %-8.4f
        param_tags{j}, best_p(j), param_mean(j), param_std(j), ...
        param_ci(1,j), param_ci(2,j), param_min(j), param_max(j));
end
```

```
  beta     | 0.6423     | 0.6429     | 0.0532     | [0.5515  , 0.7551  ] | [0.5053  , 0.8537  ]
  alpha    | 1088.6350  | 1079.6253  | 73.8391    | [945.7832, 1230.1044] | [866.1037, 1354.7886]
  delta    | 0.0486     | 0.0460     | 0.0019     | [0.0423  , 0.0495  ] | [0.0406  , 0.0513  ]
```

```
% Bootstrap trajectories for confidence bands
sim_prev_boot  = NaN(length(times), n_valid);
sim_death_boot = NaN(length(times), n_valid);
for i = 1:n_valid
    pit  = param_boot(i,:);
    dpit = k * pit(3);
    try
        [~, xi] = ode45(@(t,y) ode_macro(t,y,pit,fixed_params), times, x0);
        sim_prev_boot(:,i)  = xi(:,3) + xi(:,4);
        sim_death_boot(:,i) = pit(3).*xi(:,3) + dpit.*xi(:,4);
    catch
    end
end

prev_lo  = prctile(sim_prev_boot',  2.5)';
prev_hi  = prctile(sim_prev_boot', 97.5)';
death_lo = prctile(sim_death_boot', 2.5)';
death_hi = prctile(sim_death_boot',97.5)';

%% ========================================================================
%  SECTION 7: BASIC REPRODUCTION NUMBER R0
% ========================================================================
R0_best = compute_R0(best_p(1), best_p(3), Lambda, nu, k, gamma_art);

R0_boot = zeros(n_valid, 1);
for i = 1:n_valid
    R0_boot(i) = compute_R0(param_boot(i,1), param_boot(i,3), Lambda, nu, k, gamma_art
end
R0_ci = prctile(R0_boot, [2.5, 97.5]);

fprintf('\n=====================================\n');
```

```
=====================================
```

```
fprintf('  BASIC REPRODUCTION NUMBER R0          \n');
```

```
  BASIC REPRODUCTION NUMBER R0
```

```
fprintf('========================================\n');
```

```
========================================
```

```
fprintf('  R0 (best estimate) = %.4f\n',          R0_best);
```

```
  R0 (best estimate) = 3.7992
```

```
fprintf('  R0 mean (bootstrap)= %.4f\n',          mean(R0_boot));
```

```
  R0 mean (bootstrap)= 3.9561
```

```
fprintf('  R0 95%% CI          = [%.4f, %.4f]\n', R0_ci(1), R0_ci(2));
```

```
  R0 95% CI         = [3.3187, 4.7044]
```

```
fprintf('  R0 range           = [%.4f, %.4f]\n',  min(R0_boot), max(R0_boot));
```

```
  R0 range          = [2.9997, 5.3275]
```

```
fprintf('  Status             : %s\n', ...
    ternary(R0_best > 1, 'ENDEMIC (R0 > 1)', 'DISEASE-FREE (R0 < 1)'));
```

```
  Status            : ENDEMIC (R0 > 1)
```

```
%% ========================================================================
%  SECTION 8: CALIBRATION FIGURES
% ========================================================================
sc_p = 1e3;   % display scale for prevalence (x10^-3)
sc_d = 1e4;   % display scale for deaths     (x10^-4)

% --- Figure: Model Fit Prevalence ---
f_prev = new_fig();
fill_band(year_obs, prev_lo*sc_p, prev_hi*sc_p);
plot(year_obs, sim_prev*sc_p,  'k-',  'LineWidth', 2, 'DisplayName', 'Model');
plot(year_obs, data_prev*sc_p, 'ro',  'MarkerFaceColor','r','MarkerSize',6,...
     'DisplayName','UNAIDS Data');
xlabel('Year'); ylabel('HIV Infection Proportion (x 10^{-3})');
legend('Location','northwest');
fmt_ax();

% --- Figure: Model Fit Mortality ---
f_death = new_fig();
fill_band(year_obs, death_lo*sc_d, death_hi*sc_d);
plot(year_obs, sim_death*sc_d, 'k-',  'LineWidth', 2, 'DisplayName','Model');
plot(year_obs, data_death*sc_d,'ro',  'MarkerFaceColor','r','MarkerSize',6,...
     'DisplayName','UNAIDS Data');
xlabel('Year'); ylabel('AIDS Mortality Proportion (x 10^{-4})');
legend('Location','northwest');
fmt_ax();

% --- Figure: R0 bootstrap distribution ---
f_R0 = new_fig();
```

```
bin_width = 0.05;
min_r0 = floor(min(R0_boot) * 100) / 100;
max_r0 = ceil(max(R0_boot) * 100) / 100;
edges = min_r0 : bin_width : max_r0;
if length(edges) < 2
    edges = [min(R0_boot)-0.01, max(R0_boot)+0.01];
end
histogram(R0_boot, 'BinEdges', edges, 'FaceColor','#0072BD','EdgeColor','white');
xline(R0_best, 'r--','LineWidth',2,...
      'Label', sprintf('Best R0 = %.3f', R0_best));
xline(1, 'k:','LineWidth',2,'Label','Threshold (R0 = 1)');
xlabel('Basic Reproduction Number (R0)'); ylabel('Bootstrap Frequency');
fmt_ax();

%% ========================================================================
%  SECTION 9: EXTENDED MACRO PROJECTION
% ========================================================================
t_mac    = (0:400)';
year_mac = year_base + t_mac;

[~, x_mac] = ode45(@(t,y) ode_macro(t,y,best_p,fixed_params), t_mac, x0_mac);
I_eff_mac = x_mac(:,3) + k_inf * x_mac(:,4);

%% ========================================================================
%  SECTION 10: INSURANCE PRODUCT & ACTUARIAL SETTINGS
% ========================================================================
r        = 0.06;
bi_IDR   = 18e6;
bd_IDR   = 50e6;
N_p0     = 10000;
y0_polis = [1; 0; 0];

tau0_2026 = 2026 - year_base;
T_term = 20;
T_inf  = 300;

%% ========================================================================
%  SECTION 11: COHORT DEMOGRAPHIC TRANSITIONS
% ========================================================================
T_demo  = 150;
dt_demo = 0.25;
t_demo  = (0:dt_demo:T_demo)';

[~, y_demo] = ode45( ...
    @(t,y) ode_cohort(t,y,best_p,fixed_params,t_mac,I_eff_mac,tau0_2026), ...
    t_demo, y0_polis);

ps_demo = max(0, y_demo(:,1));
pi_demo = max(0, y_demo(:,2));
pd_demo = max(0, y_demo(:,3));

Sp_abs = N_p0 * ps_demo;
Ip_abs = N_p0 * pi_demo;
Dp_abs = N_p0 * pd_demo;
```

```
% --- Sp Trajectory ---
f_cohort_Sp = new_fig();
plot(t_demo, Sp_abs, 'b-', 'LineWidth', 2);
xlabel('Policy Duration (years)'); ylabel('Number of Policyholders');
fmt_ax();

% --- Ip Trajectory ---
f_cohort_Ip = new_fig();
plot(t_demo, Ip_abs, 'r-', 'LineWidth', 2);
xlabel('Policy Duration (years)'); ylabel('Number of Policyholders');
fmt_ax();

% --- Dp Trajectory ---
f_cohort_Dp = new_fig();
plot(t_demo, Dp_abs, 'k-', 'LineWidth', 2);
xlabel('Policy Duration (years)'); ylabel('Cumulative Deaths');
fmt_ax();

fprintf('\n=========================================\n');
```

```
=====================================
```

```
fprintf('  COHORT DEMOGRAPHIC TRANSITIONS (tau0=2026, N_p(0)=%d)\n', N_p0);
```

```
  COHORT DEMOGRAPHIC TRANSITIONS (tau0=2026, N_p(0)=10000)
```

```
fprintf('=========================================\n');
```

```
=====================================
```

```
[Ip_peak, idx_peak] = max(Ip_abs);
fprintf('  Peak I_p (point-in-time prevalence) = %.1f policyholders at t = %.2f years\n
    Ip_peak, t_demo(idx_peak));
```

```
  Peak I_p (point-in-time prevalence) = 43.2 policyholders at t = 69.75 years
```

```
fprintf('  Terminal D_p (cumulative, t=%d yrs)  = %.1f policyholders\n', T_demo, Dp_abs
```

```
  Terminal D_p (cumulative, t=150 yrs)  = 156.4 policyholders
```

```


%% ========================================================================
%  SECTION 12: ACTUARIAL VALUATION AT tau0 = 2026
% ========================================================================
dt_term = 0.05;
dt_inf  = 0.5;

[t20,  a_ps20,  a_pi20,  A_pi20,  A_pd20 ] = valuate_cohort(tau0_2026, T_term, dt_term
    best_p, fixed_params, t_mac, I_eff_mac, y0_polis, r);
[tinf, a_psInf, a_piInf, A_piInf, A_pdInf] = valuate_cohort(tau0_2026, T_inf,  dt_inf,
    best_p, fixed_params, t_mac, I_eff_mac, y0_polis, r);

% --- T = 20 (term) ---
APV_ps_20  = a_ps20(end);
APV_pi_20  = a_pi20(end);
```

```
APV_Api_20 = A_pi20(end);
APV_Apd_20 = A_pd20(end);

R1_20 = APV_pi_20  / APV_ps_20;
R2_20 = APV_Api_20 / APV_ps_20;
Rd_20 = APV_Apd_20 / APV_ps_20;
pi1_20 = R1_20;
pi2_20 = R2_20;
pi3_20 = R1_20 + Rd_20;
pi4_20 = R2_20 + Rd_20;

% --- T -> infinity (whole life) ---
APV_ps_inf  = a_psInf(end);
APV_pi_inf  = a_piInf(end);
APV_Api_inf = A_piInf(end);
APV_Apd_inf = A_pdInf(end);

R1_inf = APV_pi_inf  / APV_ps_inf;
R2_inf = APV_Api_inf / APV_ps_inf;
Rd_inf = APV_Apd_inf / APV_ps_inf;
pi1_inf = R1_inf;
pi2_inf = R2_inf;
pi3_inf = R1_inf + Rd_inf;
pi4_inf = R2_inf + Rd_inf;

% --- Closed-form whole-life validation ---
pi1_closed = (1 - (r+mu)*APV_ps_inf) / ((r+mu+delta_p)*APV_ps_inf);
pi2_closed = (r+mu+delta_p) * pi1_closed;
pi3_closed = (1+delta_p) * pi1_closed;
pi4_closed = (r+mu+2*delta_p) * pi1_closed;

fprintf('\n======================================================================\n')
```

```
======================================================================
```

```
fprintf('  TABLE: UNIT-BASIS DECOMPOSITION (tau0=2026, r=%.2f) \n', r);
```

```
  TABLE: UNIT-BASIS DECOMPOSITION (tau0=2026, r=0.06)
```

```
fprintf('======================================================================\n');
```

```
======================================================================
```

```
fprintf('  %-24s | %-12s | %-10s | %-12s | %-10s\n', ...
    'Component / Scenario','APV(T=20)','Rate(T=20)','APV(T->inf)','Rate(T->inf)');
```

```
  Component / Scenario     | APV(T=20)    | Rate(T=20) | APV(T->inf)  | Rate(T->inf)
```

```
fprintf('  %s\n', repmat('-',1,78));
```

```
  ------------------------------------------------------------------------------
```

```
fprintf('  %-24s | %-12.6e | %-10s | %-12.6e | %-10s\n', ...
    'Healthy Annuity', APV_ps_20, '---', APV_ps_inf, '---');
```

```
  Healthy Annuity          | 1.119805e+01 | ---        | 1.540160e+01 | ---
```

```
fprintf('  %-24s | %-12.6e | %-10.6f | %-12.6e | %-10.6f\n', ...
    'Continuous Care', APV_pi_20, R1_20, APV_pi_inf, R1_inf);
```

```
  Continuous Care          | 1.425858e-02 | 0.001273   | 3.181465e-02 | 0.002066
```

```
fprintf('  %-24s | %-12.6e | %-10.6f | %-12.6e | %-10.6f\n', ...
    'Lump-Sum Care',   APV_Api_20, R2_20, APV_Api_inf, R2_inf);
```

```
  Lump-Sum Care            | 2.137557e-03 | 0.000191   | 2.987422e-03 | 0.000194
```

```
fprintf('  %-24s | %-12.6e | %-10.6f | %-12.6e | %-10.6f\n', ...
    'Death Benefit',   APV_Apd_20, Rd_20, APV_Apd_inf, Rd_inf);
```

```
  Death Benefit            | 4.157921e-04 | 0.000037   | 9.277420e-04 | 0.000060
```

```
fprintf('  %s\n', repmat('-',1,78));
```

```
  ------------------------------------------------------------------------------
```

```
fprintf('  %-24s | %-12.6e | %-10.6f | %-12.6e | %-10.6f\n', ...
    'Scenario 1 (Cont. Care)', APV_pi_20, pi1_20, APV_pi_inf, pi1_inf);
```

```
  Scenario 1 (Cont. Care)  | 1.425858e-02 | 0.001273   | 3.181465e-02 | 0.002066
```

```
fprintf('  %-24s | %-12.6e | %-10.6f | %-12.6e | %-10.6f\n', ...
    'Scenario 2 (Lump Care)', APV_Api_20, pi2_20, APV_Api_inf, pi2_inf);
```

```
  Scenario 2 (Lump Care)   | 2.137557e-03 | 0.000191   | 2.987422e-03 | 0.000194
```

```
fprintf('  %-24s | %-12.6e | %-10.6f | %-12.6e | %-10.6f\n', ...
    'Scenario 3 (Cont.+Death)', APV_pi_20+APV_Apd_20, pi3_20, APV_pi_inf+APV_Apd_inf, 
```

```
  Scenario 3 (Cont.+Death) | 1.467437e-02 | 0.001310   | 3.274239e-02 | 0.002126
```

```
fprintf('  %-24s | %-12.6e | %-10.6f | %-12.6e | %-10.6f\n', ...
    'Scenario 4 (Lump+Death)', APV_Api_20+APV_Apd_20, pi4_20, APV_Api_inf+APV_Apd_inf,
```

```
  Scenario 4 (Lump+Death)  | 2.553349e-03 | 0.000228   | 3.915164e-03 | 0.000254
```

```
fprintf('\n  Closed-form whole-life validation (numerical vs. analytical):\n');
```

```
  Closed-form whole-life validation (numerical vs. analytical):
```

```
fprintf('    pi1: numeric = %.6f | closed-form = %.6f\n', pi1_inf, pi1_closed);
```

```
    pi1: numeric = 0.002066 | closed-form = 0.002005
```

```
fprintf('    pi2: numeric = %.6f | closed-form = %.6f\n', pi2_inf, pi2_closed);
```

```
    pi2: numeric = 0.000194 | closed-form = 0.000188
```

```
fprintf('    pi3: numeric = %.6f | closed-form = %.6f\n', pi3_inf, pi3_closed);
```

```
    pi3: numeric = 0.002126 | closed-form = 0.002064
```

```
fprintf('    pi4: numeric = %.6f | closed-form = %.6f\n', pi4_inf, pi4_closed);
```

```
    pi4: numeric = 0.000254 | closed-form = 0.000247
```

```
%% ========================================================================
%  SECTION 13: REAL IDR PREMIUM COMPARISON
% ========================================================================
P1_20_IDR  = R1_20*bi_IDR/12;
P2_20_IDR  = R2_20*bi_IDR/12;
P3_20_IDR  = (R1_20*bi_IDR + Rd_20*bd_IDR)/12;
P4_20_IDR  = (R2_20*bi_IDR + Rd_20*bd_IDR)/12;

P1_inf_IDR = R1_inf*bi_IDR/12;
P2_inf_IDR = R2_inf*bi_IDR/12;
P3_inf_IDR = (R1_inf*bi_IDR + Rd_inf*bd_IDR)/12;
P4_inf_IDR = (R2_inf*bi_IDR + Rd_inf*bd_IDR)/12;

markup1 = (P1_inf_IDR - P1_20_IDR)/P1_20_IDR*100;
markup2 = (P2_inf_IDR - P2_20_IDR)/P2_20_IDR*100;
markup3 = (P3_inf_IDR - P3_20_IDR)/P3_20_IDR*100;
markup4 = (P4_inf_IDR - P4_20_IDR)/P4_20_IDR*100;

fprintf('\n========================================================================\n')
```

```
========================================================================
```

```
fprintf('  TABLE: REAL-TERM MONTHLY PREMIUM (IDR) \n');
```

```
  TABLE: REAL-TERM MONTHLY PREMIUM (IDR)
```

```
fprintf('  bi = bL = Rp %s | bd = Rp %s\n', num2str(bi_IDR), num2str(bd_IDR));
```

```
  bi = bL = Rp 18000000 | bd = Rp 50000000
```

```
fprintf('========================================================================\n');
```

```
========================================================================
```

```
fprintf('  %-34s | %-16s | %-16s | %-10s\n', ...
    'Scenario','Term (T=20)','Whole-Life','Markup %');
```

```
  Scenario                           | Term (T=20)      | Whole-Life       | Markup %
```

```
fprintf('  %s\n', repmat('-',1,82));
```

```
  ----------------------------------------------------------------------------------
```

```
fprintf('  %-34s | Rp %-13.2f | Rp %-13.2f | %-10.2f\n', ...
    '1: Continuous Care Annuity',          P1_20_IDR, P1_inf_IDR, markup1);
```

```
  1: Continuous Care Annuity         | Rp 1909.96       | Rp 3098.51       | 62.23
```

```
fprintf('  %-34s | Rp %-13.2f | Rp %-13.2f | %-10.2f\n', ...
    '2: Lump-Sum Care Benefit',            P2_20_IDR, P2_inf_IDR, markup2);
```

```
  2: Lump-Sum Care Benefit           | Rp 286.33        | Rp 290.95        | 1.61
```

```
fprintf('  %-34s | Rp %-13.2f | Rp %-13.2f | %-10.2f\n', ...
    '3: Continuous Care + Death Lump Sum', P3_20_IDR, P3_inf_IDR, markup3);
```

```
  3: Continuous Care + Death Lump Sum | Rp 2064.68       | Rp 3349.49       | 62.23
```

```
fprintf('  %-34s | Rp %-13.2f | Rp %-13.2f | %-10.2f\n', ...
    '4: Lump-Sum Care + Death Lump Sum',   P4_20_IDR, P4_inf_IDR, markup4);
```

```
  4: Lump-Sum Care + Death Lump Sum  | Rp 441.04        | Rp 541.94        | 22.88
```

```
%% ========================================================================
%  SECTION 14: SOLVENCY BOUND & PREMIUM ADJUSTMENT   [feeds 4.4, Table premium_adjustme
% ========================================================================
tau0_years_solv = [2000, 2010, 2020, 2026];
n_solv          = length(tau0_years_solv);
scenarios_solv  = [3, 4];

pi_eq_tab   = zeros(n_solv, 2);   % columns: [Scenario 3, Scenario 4]
pi_star_tab = zeros(n_solv, 2);
tsup_tab    = zeros(n_solv, 2);
margin_tab  = zeros(n_solv, 2);

dt_wl = 0.25;   % integration step for the whole-life (T_inf) horizon

% Storage for the reserve-trajectory figure (tau0 = 2026 case only)
t_2026 = []; a_ps_2026 = []; B3_2026 = []; B4_2026 = [];
pi_eq3_2026 = 0; pi_eq4_2026 = 0;

fprintf('\n========================================================================\n')
```

```
========================================================================
```

```
fprintf('  TABLE: EQUIVALENCE vs. SOLVENCY-ADJUSTED PREMIUM — Scenarios 3 & 4\n');
```

```
  TABLE: EQUIVALENCE vs. SOLVENCY-ADJUSTED PREMIUM — Scenarios 3 & 4
```

```
fprintf('  feeds "premium_adjustment" (T -> infinity, proxy T_inf=%d)\n', T_inf);
```

```
  feeds "premium_adjustment" (T -> infinity, proxy T_inf=300)
```

```
fprintf('========================================================================\n');
```

```
========================================================================
```

```
fprintf('  %-12s | %-4s | %-16s | %-16s | %-20s | %-10s\n', ...
    'Entry Year','Scen','Equivalence pi','Adjusted pi*','Supremum Location','Margin(%)
```

```
  Entry Year   | Scen | Equivalence pi   | Adjusted pi*     | Supremum Location    | Margin(%)
```

```
fprintf('  %s\n', repmat('-',1,90));
```

```
  ------------------------------------------------------------------------------------------
```

```
for j = 1:n_solv
    tau0_j = tau0_years_solv(j) - year_base;

    [t_s, a_ps_s, a_pi_s, A_pi_s, A_pd_s] = valuate_cohort(tau0_j, T_inf, dt_wl, ...
        best_p, fixed_params, t_mac, I_eff_mac, y0_polis, r);
```

```
    B3_s  = a_pi_s + A_pd_s;    % Scenario 3: continuous care + death lump sum
    B4_s  = A_pi_s + A_pd_s;    % Scenario 4: lump-sum care + death lump sum
    valid = t_s > 0;

    for sidx = 1:2
        scen = scenarios_solv(sidx);
        if scen == 3
            B_s = B3_s;
        else
            B_s = B4_s;
        end

        psi = NaN(size(t_s));
        psi(valid) = B_s(valid) ./ a_ps_s(valid);

        [pi_star, idx_sup] = max(psi);
        t_sup  = t_s(idx_sup);
        pi_eq  = psi(end);
        margin = (pi_star - pi_eq) / pi_eq * 100;

        pi_eq_tab(j,sidx)   = pi_eq;
        pi_star_tab(j,sidx) = pi_star;
        tsup_tab(j,sidx)    = t_sup;
        margin_tab(j,sidx)  = margin;

        if abs(t_sup - T_inf) < dt_wl
            loc_str = 'asymptotic (t->inf)';
        else
            loc_str = sprintf('t = %.1f yrs', t_sup);
        end

        fprintf('  %-12d | %-4d | %-16.6f | %-16.6f | %-20s | %-10.2f\n', ...
            tau0_years_solv(j), scen, pi_eq, pi_star, loc_str, margin);
    end

    if tau0_years_solv(j) == 2026
        t_2026      = t_s;
        a_ps_2026   = a_ps_s;
        B3_2026     = B3_s;
        B4_2026     = B4_s;
        pi_eq3_2026 = pi_eq_tab(j,1);
        pi_eq4_2026 = pi_eq_tab(j,2);
    end
end
```

```
  2000         | 3    | 0.002090         | 0.002090         | asymptotic (t->inf)  | 0.00
  2000         | 4    | 0.000250         | 0.000250         | asymptotic (t->inf)  | 0.00
  2010         | 3    | 0.002099         | 0.002099         | asymptotic (t->inf)  | 0.00
  2010         | 4    | 0.000251         | 0.000251         | asymptotic (t->inf)  | 0.00
  2020         | 3    | 0.002095         | 0.002095         | asymptotic (t->inf)  | 0.00
  2020         | 4    | 0.000251         | 0.000251         | asymptotic (t->inf)  | 0.00
  2026         | 3    | 0.002126         | 0.002126         | asymptotic (t->inf)  | 0.00
  2026         | 4    | 0.000254         | 0.000254         | asymptotic (t->inf)  | 0.00
```

```
% --- Reserve trajectory: Scenario 3 ---
V3_2026 = exp(r*t_2026) .* (pi_eq3_2026 .* a_ps_2026 - B3_2026);
f_reserve_S3 = new_fig();
plot(t_2026, V3_2026, 'b-', 'LineWidth', 2); hold on; grid on; box on;
yline(0, 'k:', 'LineWidth', 1.5, 'HandleVisibility', 'off');
xlabel('Policy Duration (years)');
ylabel('Aggregate Reserve V_k(t) (unit basis)');
fmt_ax();

% --- Reserve trajectory: Scenario 4 ---
V4_2026 = exp(r*t_2026) .* (pi_eq4_2026 .* a_ps_2026 - B4_2026);
f_reserve_S4 = new_fig();
plot(t_2026, V4_2026, 'r-', 'LineWidth', 2); hold on; grid on; box on;
yline(0, 'k:', 'LineWidth', 1.5, 'HandleVisibility', 'off');
xlabel('Policy Duration (years)');
ylabel('Aggregate Reserve V_k(t) (unit basis)');
fmt_ax();

%% ========================================================================
%  SECTION 15: CONSEQUENCES OF PREMIUM UNDERPRICING   [feeds new subsec:underpricing]
% ========================================================================
dt_under = 0.25;

[t_u, a_ps_u, a_pi_u, ~, A_pd_u] = valuate_cohort(tau0_2026, T_inf, dt_under, ...
    best_p, fixed_params, t_mac, I_eff_mac, y0_polis, r);

B_u = bi_IDR .* a_pi_u + bd_IDR .* A_pd_u;   % Scenario-3 benefit outgo, real IDR

pi_eq_annual  = B_u(end) / a_ps_u(end);      % true whole-life Equivalence Premium (IDR
pi_sub_annual = pi_eq_annual * 0.80;         % 20%-underpriced premium (IDR/yr)

V_eq  = exp(r*t_u) .* (pi_eq_annual  .* a_ps_u - B_u);
V_sub = exp(r*t_u) .* (pi_sub_annual .* a_ps_u - B_u);

idx_neg = find(V_sub < 0, 1, 'first');
if ~isempty(idx_neg)
    t_insolvent = t_u(idx_neg);
else
    t_insolvent = NaN;
end

fprintf('\n========================================================================\n')
```

```
========================================================================
```

```
fprintf('  PREMIUM UNDERPRICING ANALYSIS (Scenario 3, tau0=2026, T->infinity, IDR)\n')
```

```
  PREMIUM UNDERPRICING ANALYSIS (Scenario 3, tau0=2026, T->infinity, IDR)
```

```
fprintf('========================================================================\n');
```

```
========================================================================
```

```
fprintf('  True whole-life Equivalence Premium : Rp %.2f/year  (Rp %.2f/month)\n', ...
    pi_eq_annual, pi_eq_annual/12);
```

```
  True whole-life Equivalence Premium : Rp 40200.51/year  (Rp 3350.04/month)
```

```
fprintf('  20%% Underpriced Premium             : Rp %.2f/year  (Rp %.2f/month)\n', .
    pi_sub_annual, pi_sub_annual/12);
```

```
  20% Underpriced Premium             : Rp 32160.41/year  (Rp 2680.03/month)
```

```
if ~isnan(t_insolvent)
    fprintf('  Underpriced reserve first turns negative at t = %.2f years\n', t_insolv
    fprintf('  (deficit deepens without bound thereafter; does not self-correct)\n');
else
    fprintf('  Underpriced reserve never turns negative over the horizon tested.\n');
end
```

```
  Underpriced reserve first turns negative at t = 32.50 years
  (deficit deepens without bound thereafter; does not self-correct)
```

```
% --- Underpricing: True Equivalence ---
f_underprice_eq = new_fig();
plot(t_u, V_eq, 'b-', 'LineWidth', 2); hold on; grid on; box on;
yline(0, 'k-', 'LineWidth', 1.5, 'HandleVisibility', 'off');
xlabel('Policy Duration (years)');
ylabel('Aggregate Reserve (IDR)');
xlim([0, 120]); ylim('auto');
fmt_ax();
ax1 = gca; ax1.YAxis.Exponent = 6; % maintain clean scientific notation

% --- Underpricing: 20 Percent Underpriced ---
f_underprice_sub = new_fig();
T_window   = 50;
idx_window = t_u <= T_window;
t_w = t_u(idx_window);
V_w = V_sub(idx_window);

hold on; grid on; box on;

if ~isnan(t_insolvent) && t_insolvent <= T_window
    x_shade = t_w(V_w < 0);
    if ~isempty(x_shade)
        y_bottom = min(V_w) * 1.1 * ones(size(x_shade));
        y_top    = zeros(size(x_shade));
        fill_band(x_shade, y_bottom, y_top);
    end
end

yline(0, 'k-', 'LineWidth', 1.5, 'HandleVisibility', 'off');
plot(t_w, V_w, 'r-', 'LineWidth', 2);

if ~isnan(t_insolvent) && t_insolvent <= T_window
    xline(t_insolvent, 'k--', 'LineWidth', 2, ...
        'Label', sprintf('Insolvency at t = %.1f', t_insolvent), ...
        'LabelVerticalAlignment', 'bottom');
end
```

```
xlabel('Policy Duration (years)');
ylabel('Aggregate Reserve (IDR)');
xlim([0, T_window]);
ylim([min(V_w)*1.1, max(V_w)*1.1]);
fmt_ax();
ax2 = gca; ax2.YAxis.Exponent = 6;

%% ========================================================================
%  SECTION 16: SENSITIVITY ANALYSIS   [feeds 4.5, Table sensitivity]
% ========================================================================
T_sens  = 20;
dt_sens = 0.05;
scenarios_sens = [3, 4];

pistar = @(scen, p_, fp_) local_pi_star(scen, tau0_2026, T_sens, dt_sens, p_, fp_, t_ma

pi_star_base = zeros(1,2);
for sidx = 1:2
    pi_star_base(sidx) = pistar(scenarios_sens(sidx), best_p, fixed_params);
end

param_names_disp = {'beta (Transmission)', 'gamma (Infectivity)', ...
                    'delta (Mortality)', 'k (ART Efficacy)'};

param_names_plot = {'\beta (Transmission)', '\gamma (Infectivity)', ...
                    '\delta (Mortality)', 'k (ART Efficacy)'};

pi_minus20 = zeros(4,2);   % rows: parameters, columns: [Scenario 3, Scenario 4]
pi_plus20  = zeros(4,2);
elasticity = zeros(4,2);

for sidx = 1:2
    scen = scenarios_sens(sidx);
    for j = 1:4
        p_lo = best_p; fp_lo = fixed_params;
        p_hi = best_p; fp_hi = fixed_params;
        switch j
            case 1  % beta
                p_lo(1) = best_p(1)*0.8;  p_hi(1) = best_p(1)*1.2;
            case 2  % gamma
                fp_lo(5) = gamma_art*0.8; fp_hi(5) = gamma_art*1.2;
            case 3  % delta
                p_lo(3) = best_p(3)*0.8;  p_hi(3) = best_p(3)*1.2;
            case 4  % k
                fp_lo(4) = k*0.8;         fp_hi(4) = k*1.2;
        end
        pi_minus20(j,sidx) = pistar(scen, p_lo, fp_lo);
        pi_plus20(j,sidx)  = pistar(scen, p_hi, fp_hi);
        delta_pct          = (pi_plus20(j,sidx) - pi_minus20(j,sidx)) / pi_star_base(si
        elasticity(j,sidx) = delta_pct / 40;   % % premium change per 1% parameter cha
    end
end

fprintf('\n========================================================================\n')
```

```
======================================================================
```

```
fprintf('  TABLE: SENSITIVITY OF SCENARIO 3 vs SCENARIO 4 TERM PREMIUM (T=20, +-20%%)\n
```

```
  TABLE: SENSITIVITY OF SCENARIO 3 vs SCENARIO 4 TERM PREMIUM (T=20, +-20%)
```

```
fprintf('======================================================================\n');
```

```
======================================================================
```

```
for sidx = 1:2
    fprintf('\n  --- Scenario %d (baseline pi* = %.6f) ---\n', scenarios_sens(sidx), p
    fprintf('  %-24s | %-14s | %-14s | %-14s | %-16s\n', ...
        'Parameter','-20%','Baseline','+20%','Elasticity (%)');
    fprintf('  %s\n', repmat('-',1,90));
    for j = 1:4
        fprintf('  %-24s | %-14.6f | %-14.6f | %-14.6f | %-16.4f\n', ...
            param_names_disp{j}, pi_minus20(j,sidx), pi_star_base(sidx), pi_plus20(j,s
    end
end
```

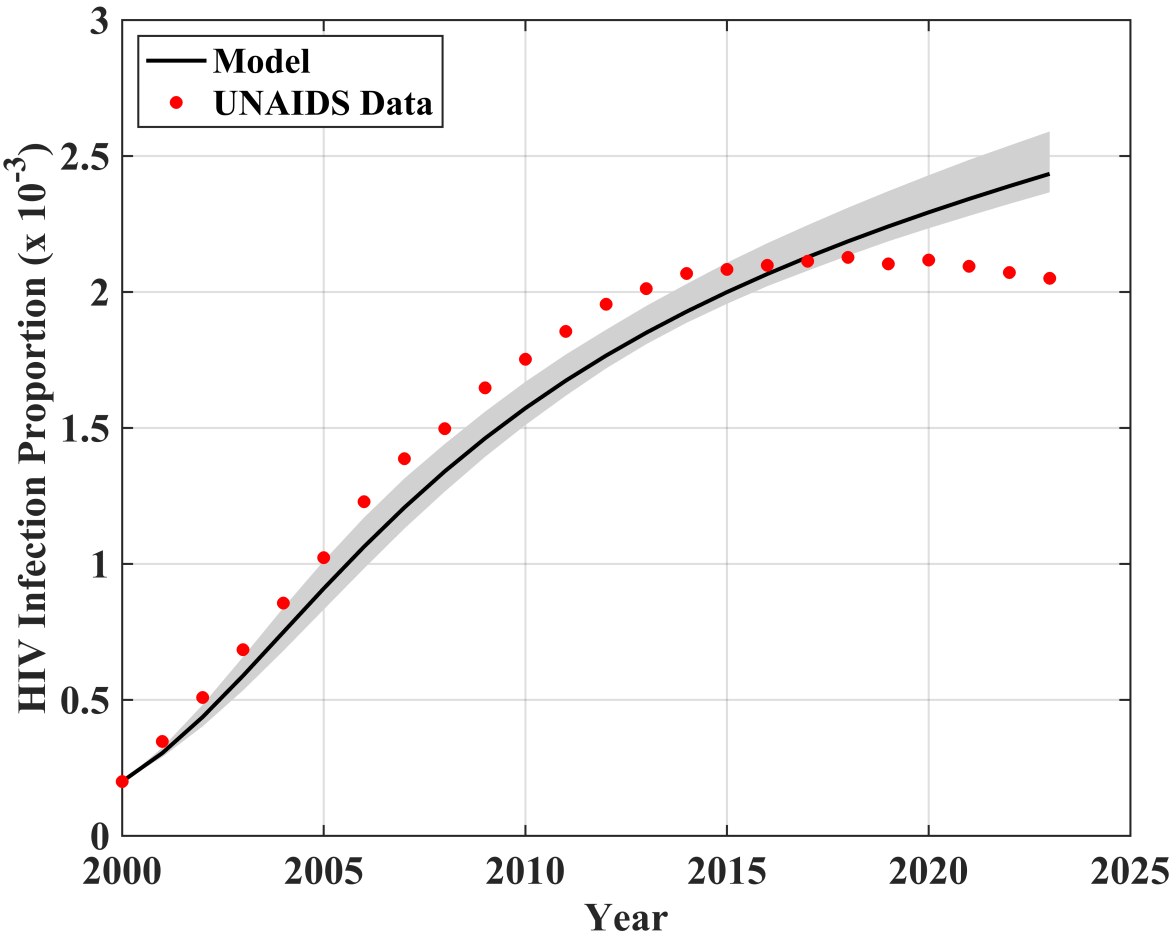
```
  --- Scenario 3 (baseline pi* = 0.001310) ---
  Parameter                | -20%           | Baseline       | +20%           | Elasticity (%)
  ------------------------------------------------------------------------------------------
  beta (Transmission)      | 0.001132       | 0.001310       | 0.001457       | 0.6197
  gamma (Infectivity)      | 0.001322       | 0.001310       | 0.001299       | -0.0440
  delta (Mortality)        | 0.001267       | 0.001310       | 0.001337       | 0.1343
  k (ART Efficacy)         | 0.001341       | 0.001310       | 0.001281       | -0.1138
  --- Scenario 4 (baseline pi* = 0.000228) ---
  Parameter                | -20%           | Baseline       | +20%           | Elasticity (%)
  ------------------------------------------------------------------------------------------
  beta (Transmission)      | 0.000197       | 0.000228       | 0.000254       | 0.6282
  gamma (Infectivity)      | 0.000230       | 0.000228       | 0.000226       | -0.0452
  delta (Mortality)        | 0.000209       | 0.000228       | 0.000245       | 0.3954
  k (ART Efficacy)         | 0.000221       | 0.000228       | 0.000235       | 0.1515
```

```
% --- Elasticity figure: Scenario 3 vs Scenario 4 (now separated) ---
pct_change_minus = (pi_minus20 - pi_star_base) ./ pi_star_base * 100;   % 4x2
pct_change_plus  = (pi_plus20  - pi_star_base) ./ pi_star_base * 100;   % 4x2

ylim_shared = [min([pct_change_minus(:); pct_change_plus(:)])*1.15, ...
               max([pct_change_minus(:); pct_change_plus(:)])*1.15];

% Elasticity Scenario 3
f_elastic_S3 = new_fig(); hold on; grid on; box on;
b3 = bar([pct_change_minus(:,1), pct_change_plus(:,1)], 'grouped');
b3(1).FaceColor = [0.20 0.45 0.85];
b3(2).FaceColor = [0.85 0.30 0.20];
set(gca, 'XTick', 1:4, 'XTickLabel', param_names_plot, 'XTickLabelRotation', 15);
ylim(ylim_shared);
yline(0, 'k-', 'LineWidth', 1, 'HandleVisibility', 'off');
ylabel('Pct. Change in Term Premium (\pi_{k}^{*}) vs. Baseline');
legend({'-20 Pct. Perturbation','+20 Pct. Perturbation'}, 'Location','best');
fmt_ax();

% Elasticity Scenario 4
```

```
f_elastic_S4 = new_fig(); hold on; grid on; box on;
b4 = bar([pct_change_minus(:,2), pct_change_plus(:,2)], 'grouped');
b4(1).FaceColor = [0.20 0.45 0.85];
b4(2).FaceColor = [0.85 0.30 0.20];
set(gca, 'XTick', 1:4, 'XTickLabel', param_names_plot, 'XTickLabelRotation', 15);
ylim(ylim_shared);
yline(0, 'k-', 'LineWidth', 1, 'HandleVisibility', 'off');
ylabel('Pct. Change in Term Premium (\pi_{k}^{*}) vs. Baseline');
legend({'-20 Pct. Perturbation','+20 Pct. Perturbation'}, 'Location','best');
fmt_ax();

%% ========================================================================
%  SECTION 17: EXPORT ALL FIGURES (300 DPI PNG)
% ========================================================================
fprintf('\nExporting 12 individual figures to PNG (300 DPI)...\n');
```

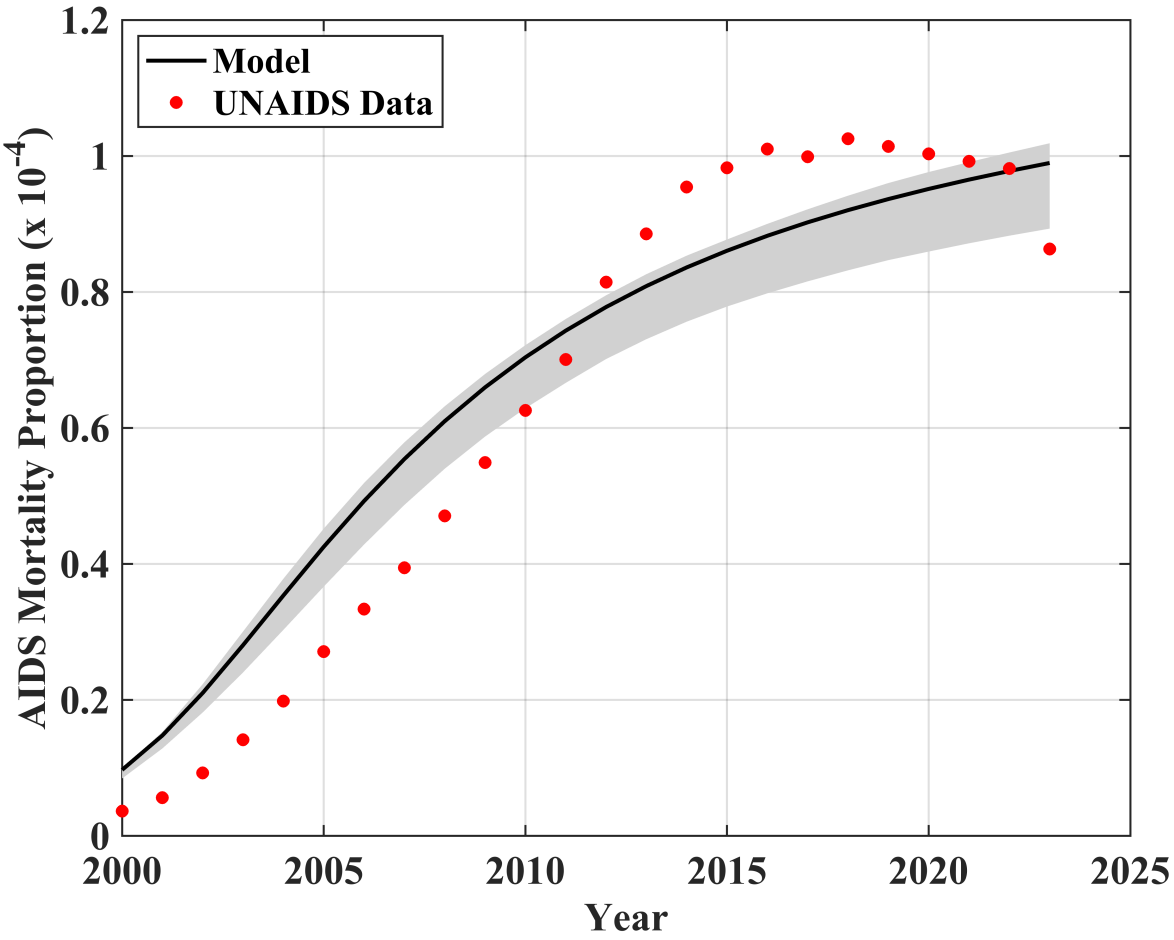
```
Exporting 12 individual figures to PNG (300 DPI)...
```

```
save_fig(f_prev,           'fig1a_prevalence');
```



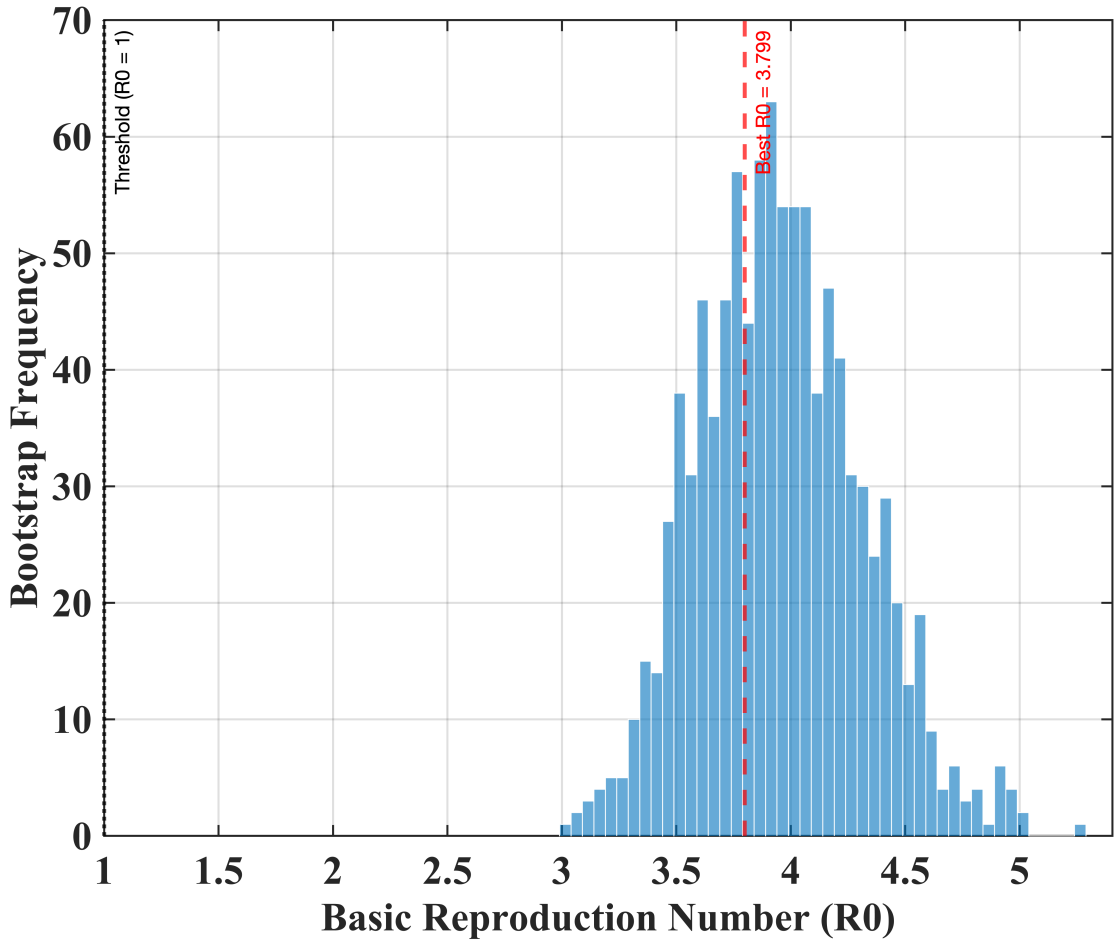

```
Saved: fig1a_prevalence.png
```

```
save_fig(f_death,          'fig1b_mortality');
```



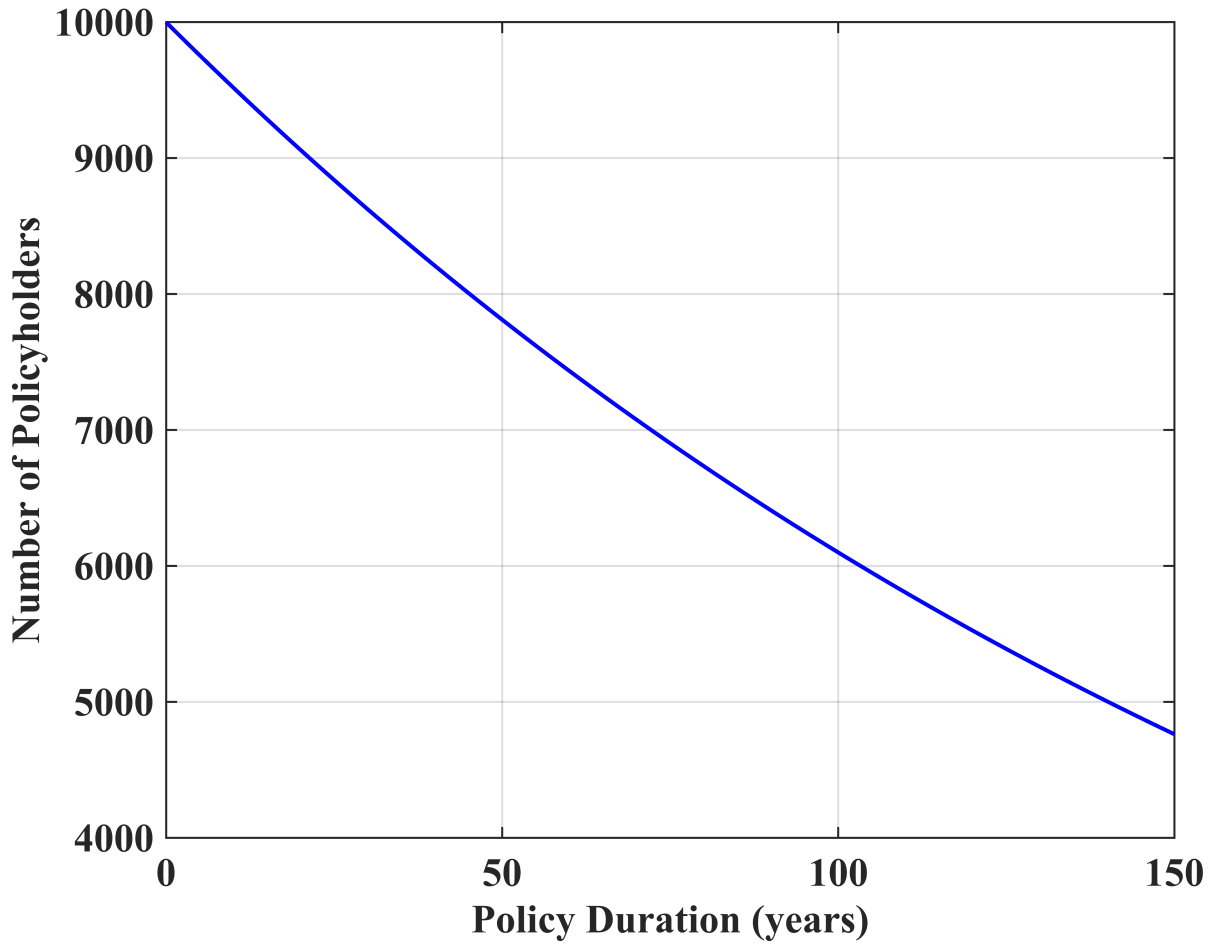

```
Saved: fig1b_mortality.png
```

```
save_fig(f_R0,              'fig2_R0');
```



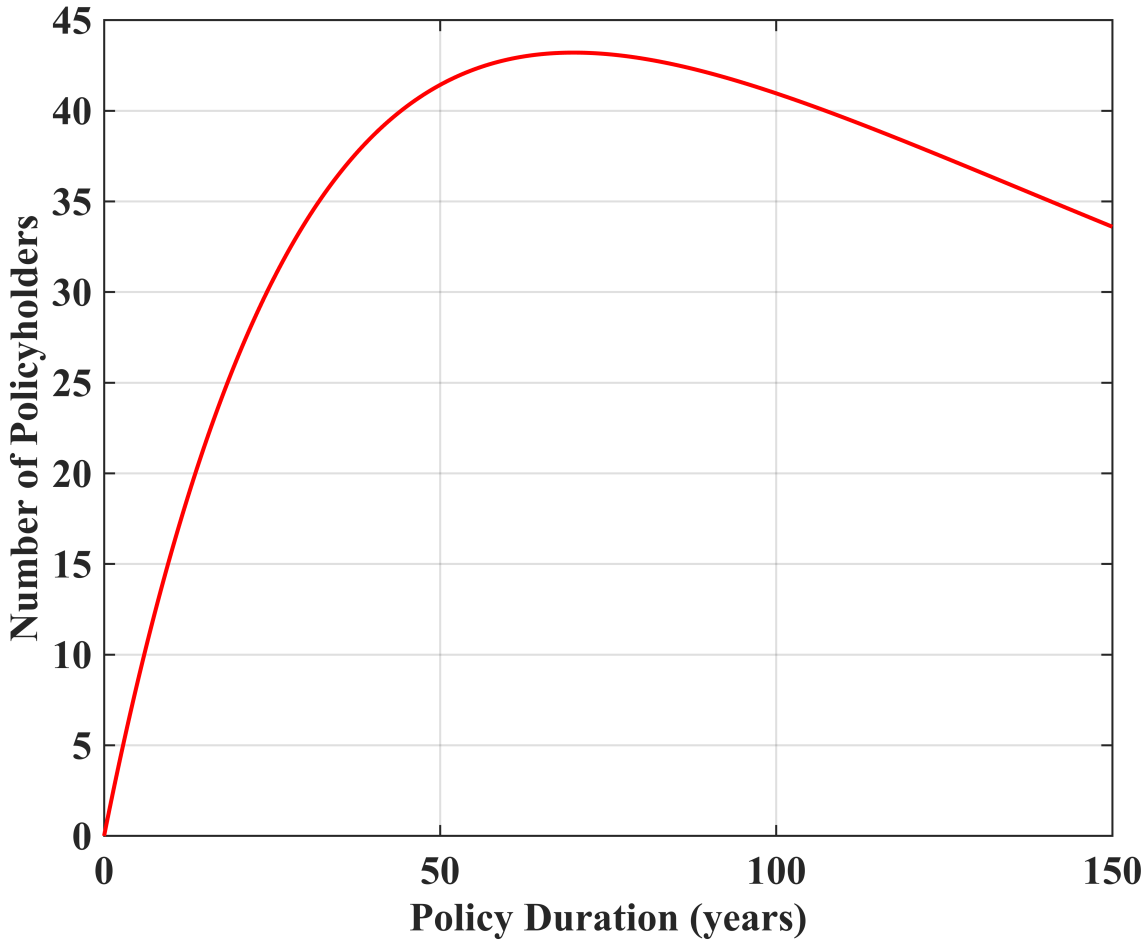

```
Saved: fig2_R0.png
```

```
save_fig(f_cohort_Sp,      'fig3a_cohort_Sp');
```



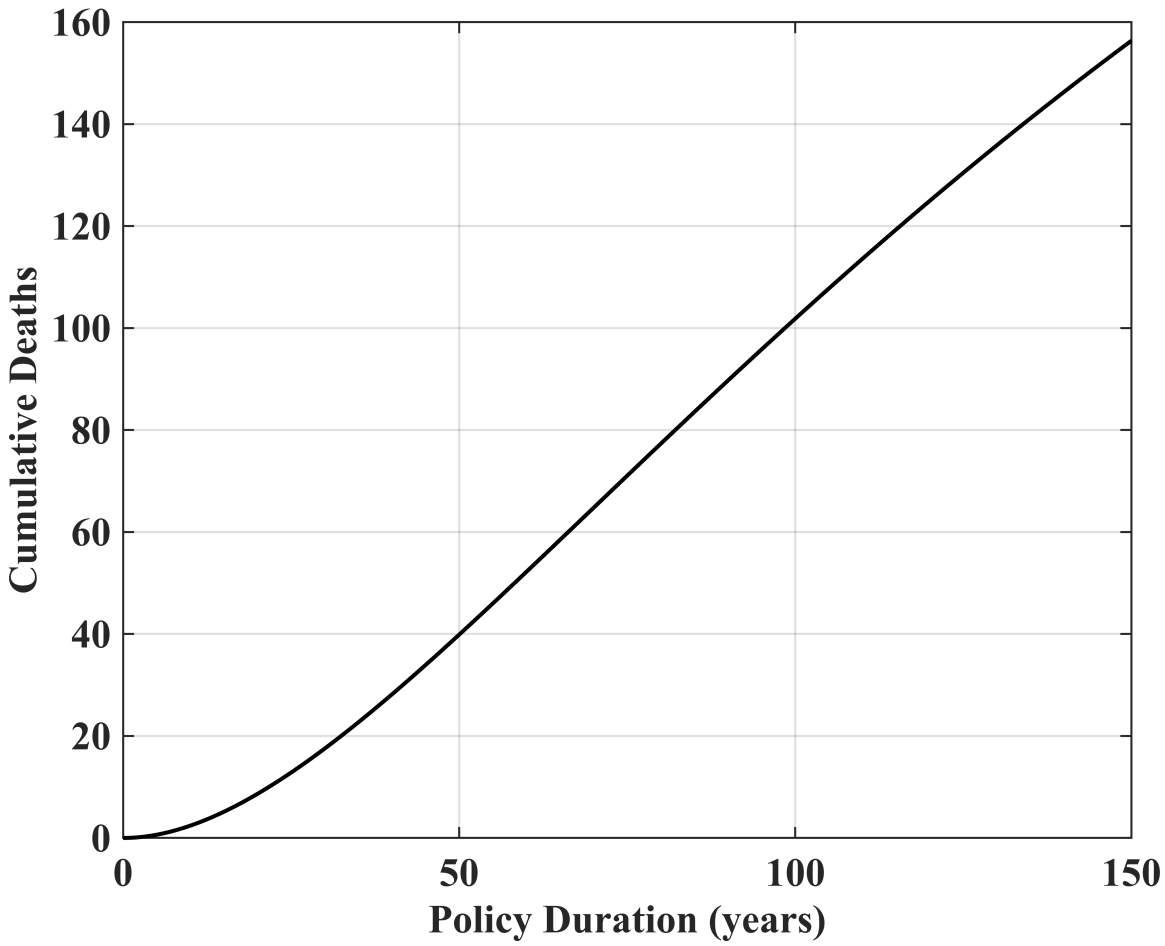

```
Saved: fig3a_cohort_Sp.png
```

```
save_fig(f_cohort_Ip,      'fig3b_cohort_Ip');
```



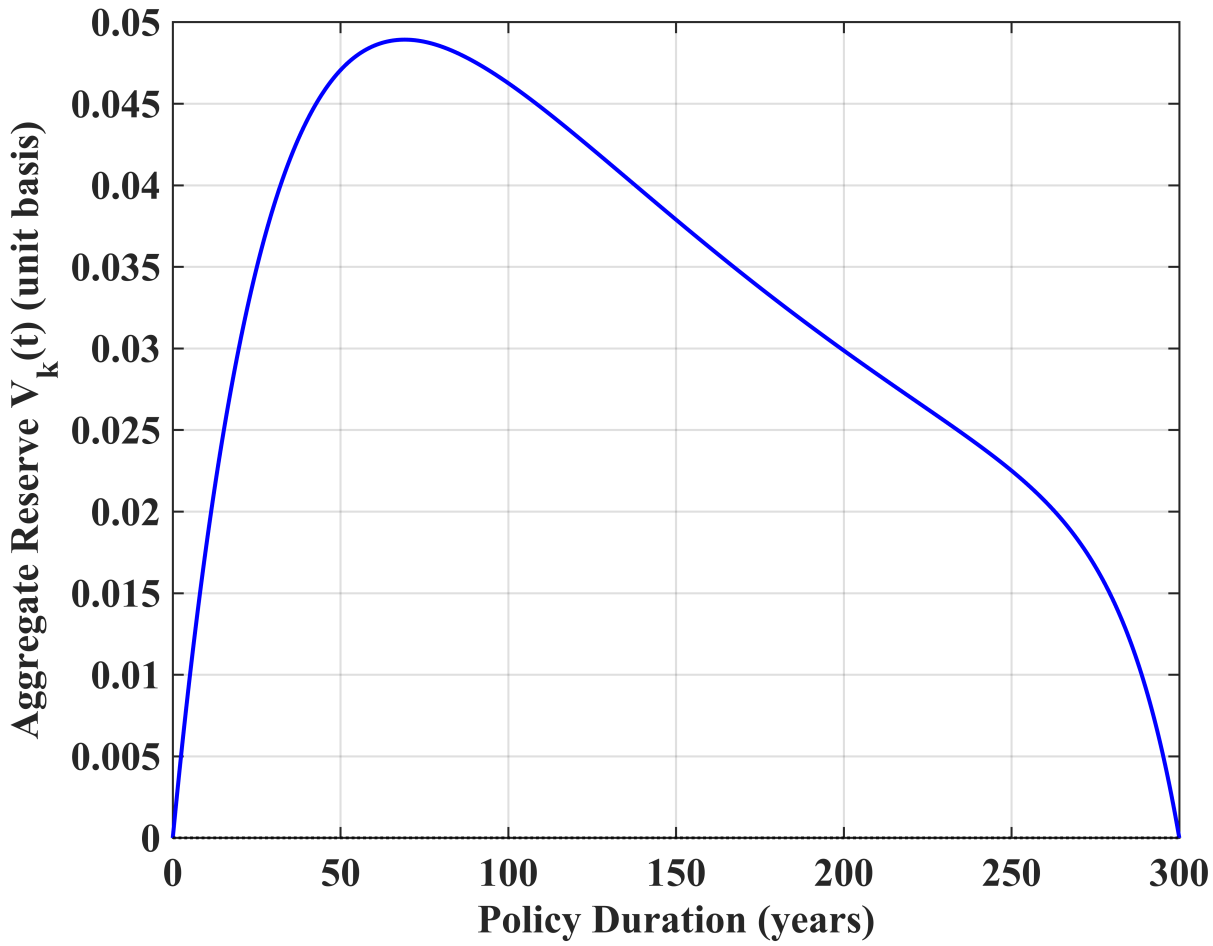

```
Saved: fig3b_cohort_Ip.png
```

```
save_fig(f_cohort_Dp,      'fig3c_cohort_Dp');
```



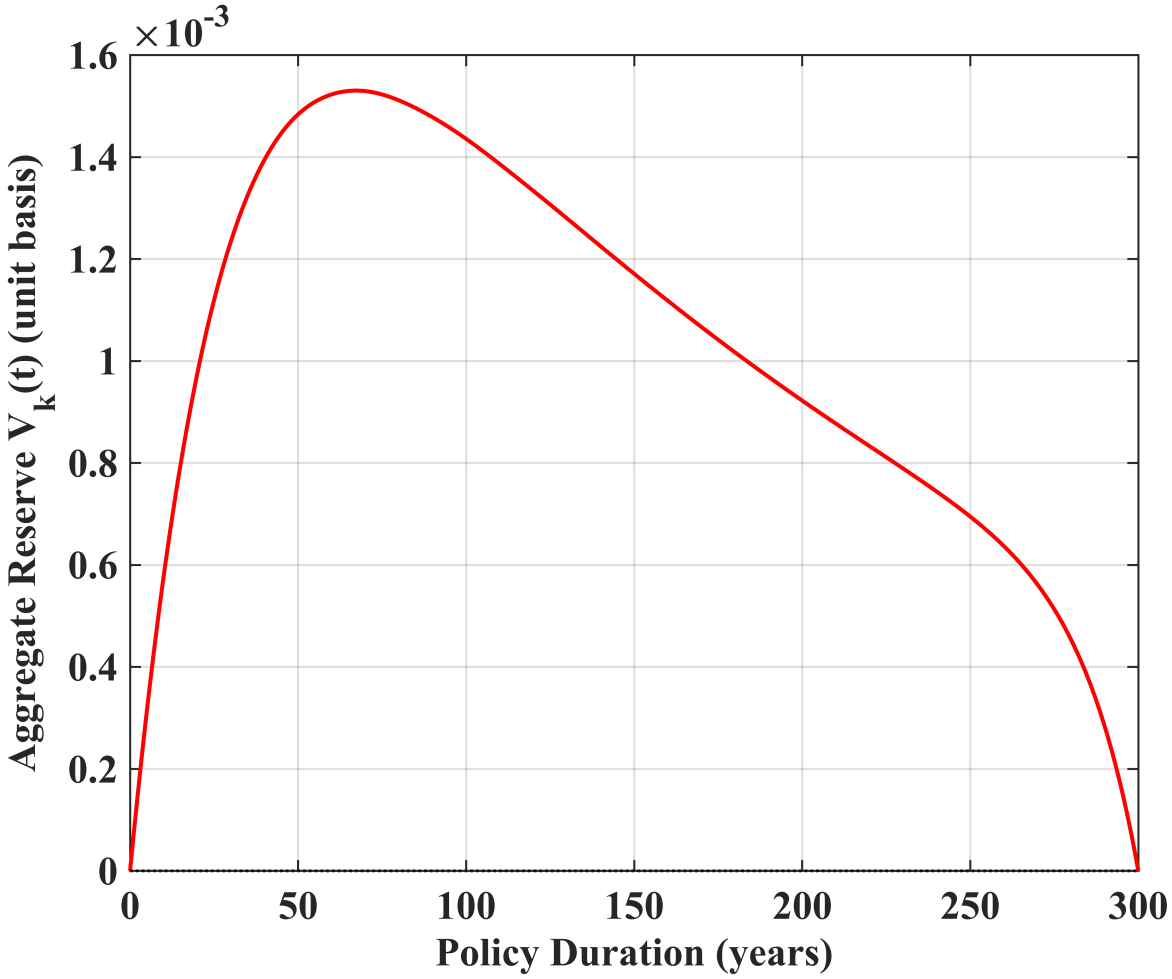

```
Saved: fig3c_cohort_Dp.png
```

```
save_fig(f_reserve_S3,     'fig4a_reserve_S3');
```



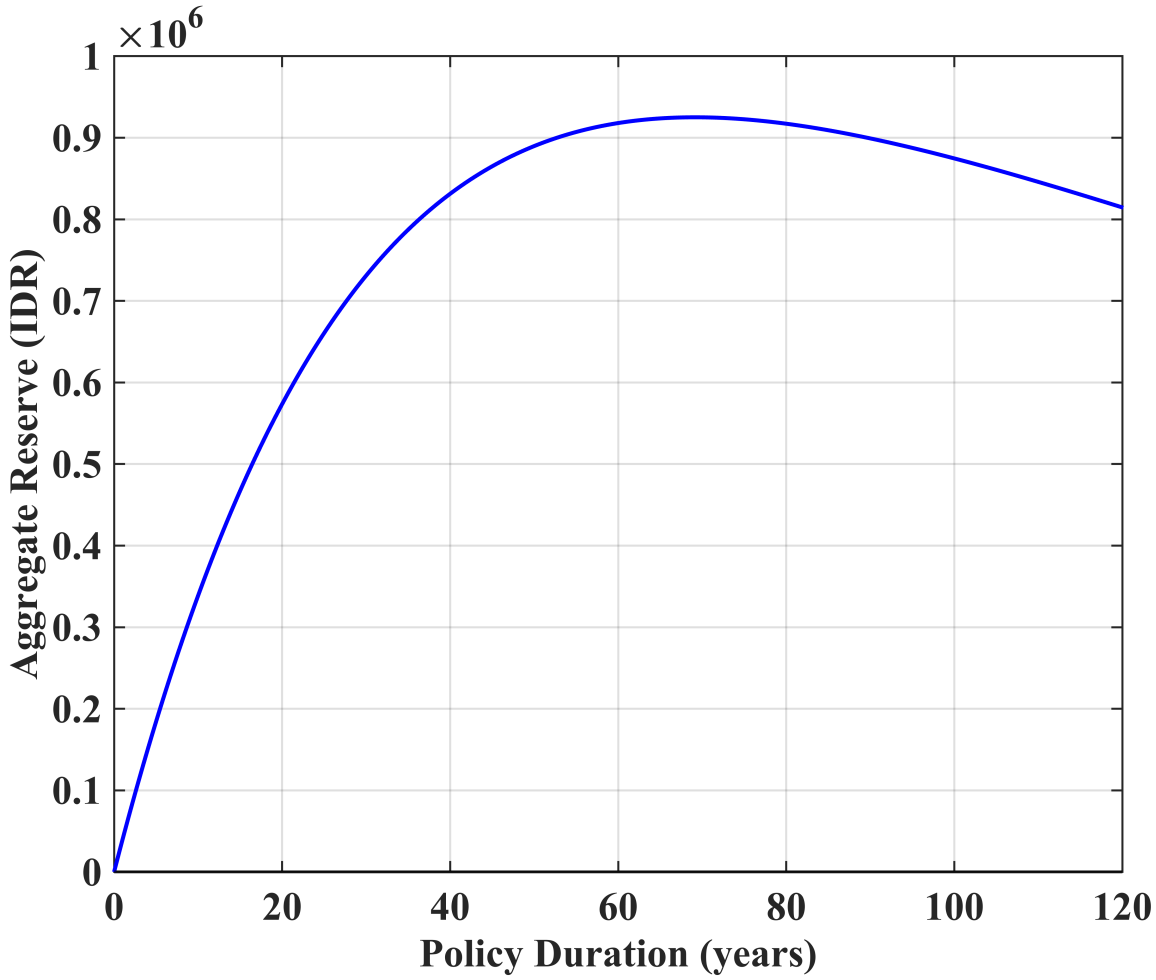

```
Saved: fig4a_reserve_S3.png
```

```
save_fig(f_reserve_S4,     'fig4b_reserve_S4');
```



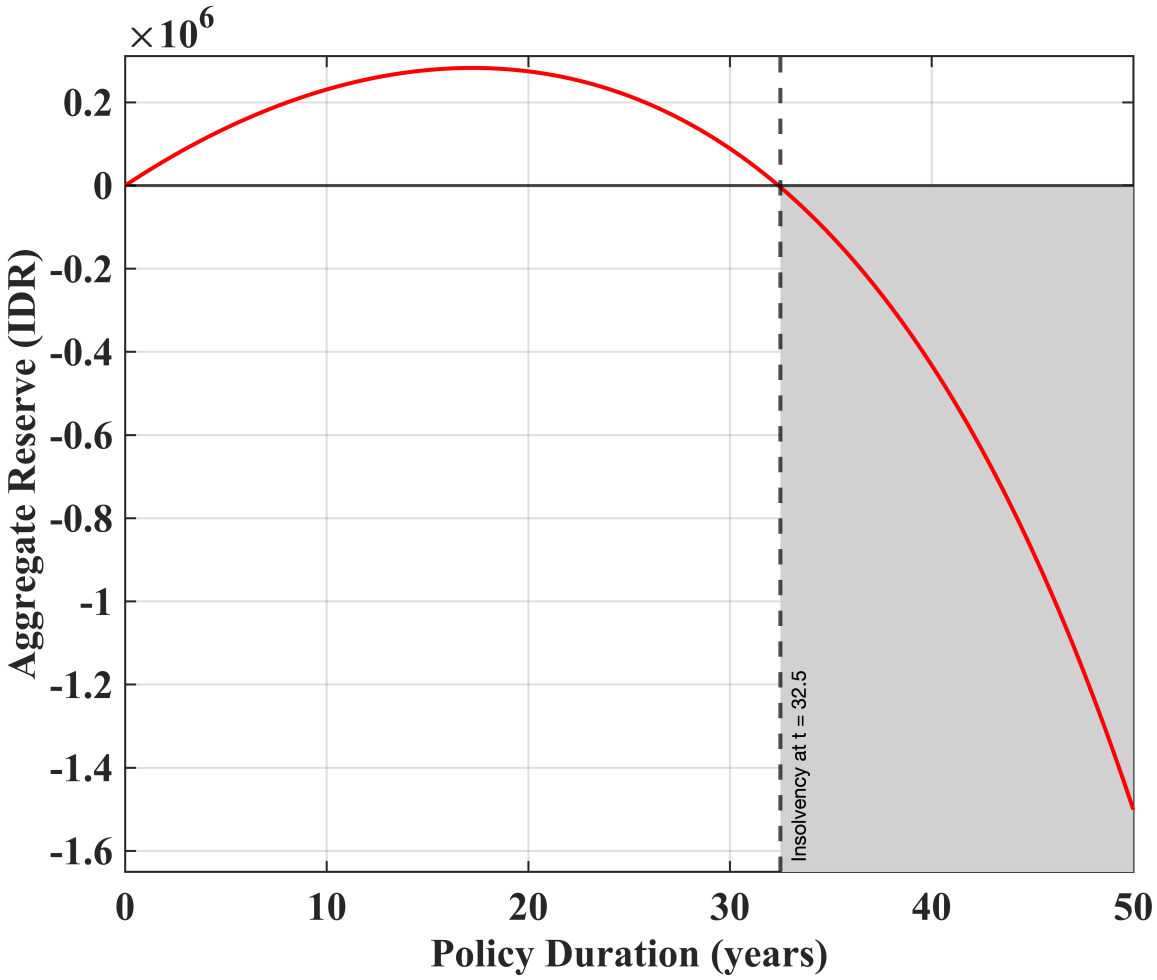

```
Saved: fig4b_reserve_S4.png
```

```
save_fig(f_underprice_eq,  'fig5a_insufficiency_eq');
```



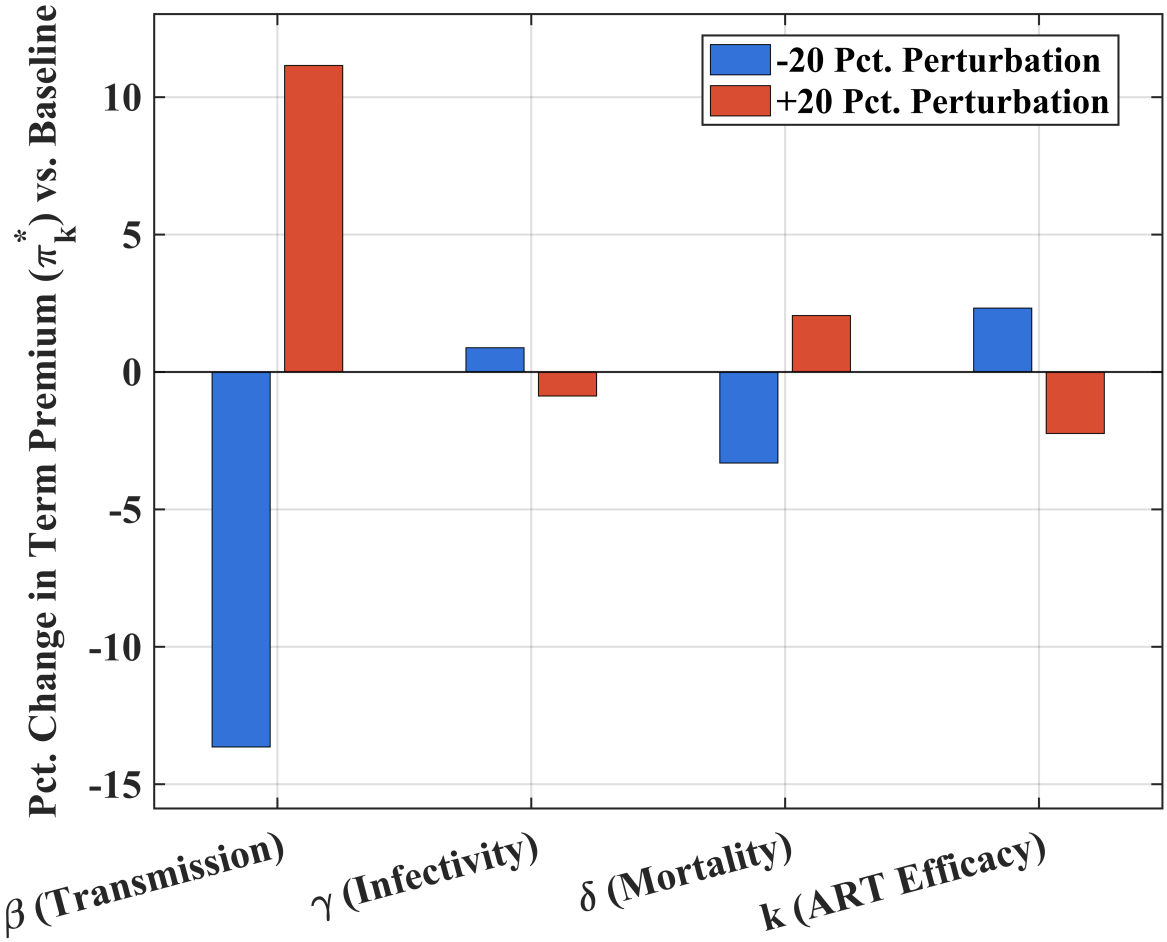

```
Saved: fig5a_insufficiency_eq.png
```

```
save_fig(f_underprice_sub, 'fig5b_insufficiency_sub');
```



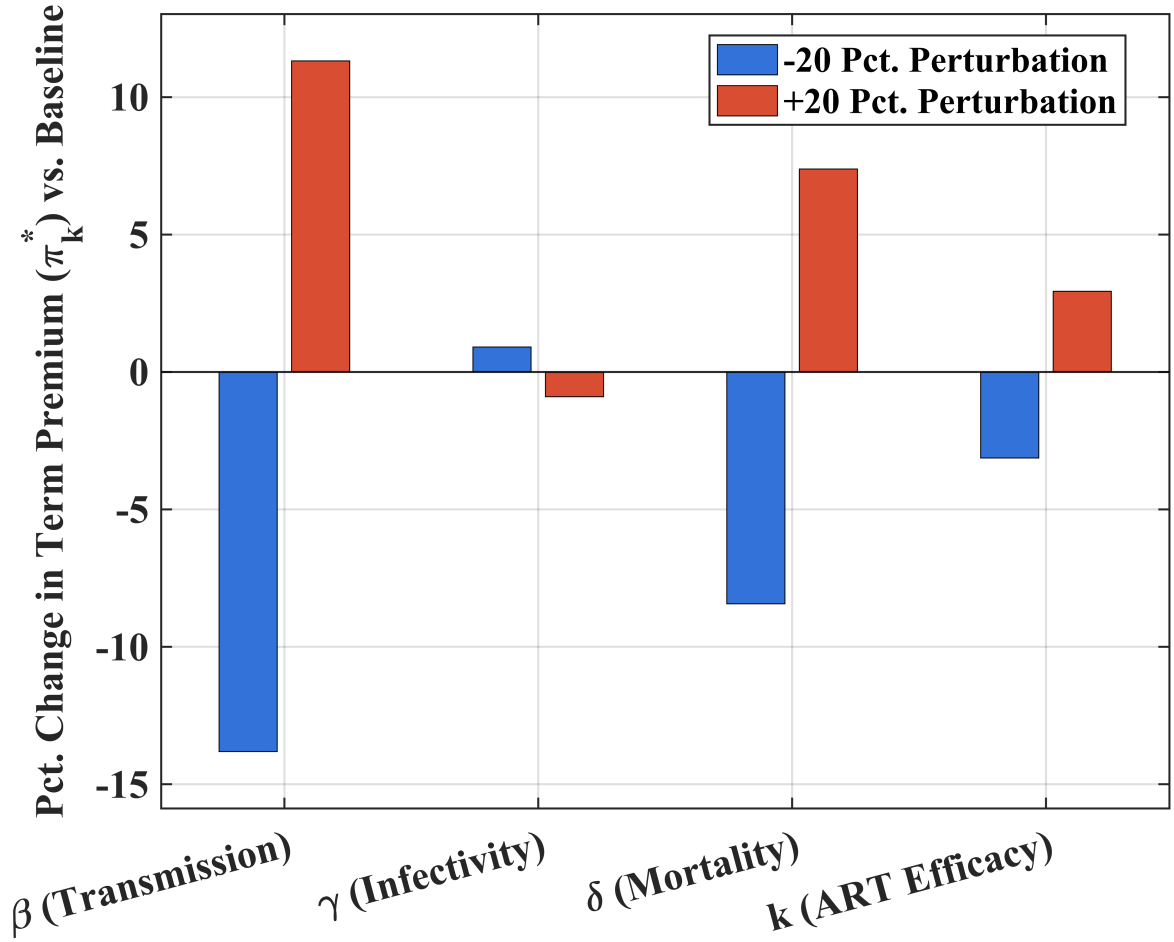

```
Saved: fig5b_insufficiency_sub.png
```

```
save_fig(f_elastic_S3,     'fig6a_sens_S3');
```




```
Saved: fig6a_sens_S3.png
```

```
save_fig(f_elastic_S4,     'fig6b_sens_S4');
```




```
Saved: fig6b_sens_S4.png
```

```
fprintf('All figures exported successfully.\n');
```

```
All figures exported successfully.
```

```
disp('========== COMPUTATION COMPLETE ==========');
```

```
========== COMPUTATION COMPLETE ==========
```

```
%% ========================================================================
%  LOCAL FUNCTIONS
% ========================================================================
```

```
function R0 = compute_R0(beta, delta, Lambda, nu, k, gamma_art)
    s_str  = Lambda / (Lambda + nu + eps);
    sp_str = nu     / (Lambda + nu + eps);
    R0 = (beta * s_str)            / (Lambda + delta) + ...
         (beta * gamma_art*sp_str) / (Lambda + k*delta);
end
```

```
function err = obj_nsse(p, times, x0, fp, data)
    try
        [~, x] = ode45(@(t,y) ode_macro(t,y,p,fp), times, x0);
    catch
        err = 1e6; return;
    end
    if any(isnan(x(:))) || any(x(:) < -1e-6)
        err = 1e6; return;
    end
    k   = fp(4);
    dp  = k * p(3);
    s_p = x(:,3) + x(:,4);
    s_d = p(3).*x(:,3) + dp.*x(:,4);
    mp  = max(data(:,1));
    md  = max(data(:,2));
    if mp < eps || md < eps, err = 1e6; return; end
    err = sum(((s_p - data(:,1))/mp).^2) + ...
          sum(((s_d - data(:,2))/md).^2);
end

function dydt = ode_macro(~, y, p, fp)
    s  = y(1);  sp = y(2);  i = y(3);  ip = y(4);

    beta   = p(1);  alpha = p(2);  delta = p(3);
    Lambda = fp(1);
    nu     = fp(3);
    k      = fp(4);
    gam    = fp(5);

    delta_p = k * delta;
    k_inf   = gam;

    I_eff = i + k_inf * ip;
    lam   = beta * exp(-alpha * I_eff) * I_eff;
    phi   = delta*i + delta_p*ip;

    ds  = Lambda*(1-s) - lam*s  - nu*s       + s*phi;
    dsp = nu*s         - lam*sp - Lambda*sp  + sp*phi;
    di  = lam*s  - (Lambda + delta  )*i  + i*phi;
    dip = lam*sp - (Lambda + delta_p)*ip + ip*phi;

    dydt = [ds; dsp; di; dip];
end

function dydt = ode_cohort(t, y, p, fp, t_mac, I_eff_mac, tau0)
    ps  = y(1);  pi_ = y(2);

    beta  = p(1);  alpha = p(2);  delta = p(3);
    mu    = fp(2);
    k     = fp(4);
    delta_p = k * delta;

    I_env = interp1(t_mac, I_eff_mac, t + tau0, 'linear', 'extrap');
    I_env = max(0, I_env);
```

```
    lam = beta * exp(-alpha * I_env) * I_env;

    dps = -(lam + mu) * ps;
    dpi =  lam * ps - (mu + delta_p) * pi_;
    dpd =  delta_p * pi_;

    dydt = [dps; dpi; dpd];
end

function [t, a_ps, a_pi, A_pi, A_pd] = valuate_cohort(tau0, T, dt, p, fp, t_mac, I_eff
    t = (0:dt:T)';
    [~, y] = ode45(@(tt,yy) ode_cohort(tt,yy,p,fp,t_mac,I_eff_mac,tau0), t, y0);

    ps = max(0, y(:,1));
    pi_ = max(0, y(:,2));

    delta   = p(3);
    k_      = fp(4);
    mu_     = fp(2);
    kdelta  = k_ * delta;

    disc = exp(-r*t);
    a_ps = cumtrapz(t, ps  .* disc);
    a_pi = cumtrapz(t, pi_ .* disc);

    A_pi = disc .* pi_ + (r + mu_ + kdelta) .* a_pi;
    A_pd = kdelta .* a_pi;
end

function pi_star = local_pi_star(scenario, tau0, T, dt, p, fp, t_mac_base, x0_mac, y0,
    [~, x_m] = ode45(@(t,y) ode_macro(t,y,p,fp), t_mac_base, x0_mac);
    I_eff = max(0, x_m(:,3) + fp(5) * x_m(:,4));

    [t, a_ps, a_pi, A_pi, A_pd] = valuate_cohort(tau0, T, dt, p, fp, t_mac_base, I_eff

    switch scenario
        case 3
            B = a_pi + A_pd;
        case 4
            B = A_pi + A_pd;
        otherwise
            error('local_pi_star: scenario must be 3 or 4');
    end

    psi = NaN(size(t));
    valid = t > 0;
    psi(valid) = B(valid) ./ a_ps(valid);

    pi_star = max(psi);
end

% ---- Utility / Display Functions ------------------------------------
```

```
function f = new_fig()
    f = figure('Position',[50+randi(300), 50+randi(200), 650, 500],'Color','w');
    hold on; grid on; box on;
end

function fill_band(x, lo, hi)
    fill([x; flipud(x)], [lo; flipud(hi)], ...
         [0.82 0.82 0.82], 'EdgeColor','none','HandleVisibility','off');
end

function fmt_ax()
    % 1. Set axis tick labels
    set(gca, 'FontName', 'Times New Roman', 'FontSize', 20, ...
             'FontWeight', 'bold', 'LineWidth', 1.0, 'TickLabelInterpreter', 'tex');

    % 2. Set X and Y axis labels
    set(get(gca,'XLabel'), 'FontName', 'Times New Roman', 'FontSize', 20, ...
                           'FontWeight', 'bold', 'Interpreter', 'tex');
    set(get(gca,'YLabel'), 'FontName', 'Times New Roman', 'FontSize', 20, ...
                           'FontWeight', 'bold', 'Interpreter', 'tex');

    % 3. Set the legend
    lgd = findobj(gca,'Type','Legend');
    if ~isempty(lgd)
        lgd.FontName = 'Times New Roman';
        lgd.FontSize = 20;
        lgd.FontWeight = 'bold';
        lgd.Interpreter = 'tex';
    end
end

function save_fig(f, fname)
    exportgraphics(f, [fname, '.jpeg'], 'Resolution', 300);
    fprintf('  Saved: %s.png\n', fname);
end

function out = ternary(cond, a, b)
    if cond, out = a; else, out = b; end
end
```